

Module 01

Kubernetes (K8s) Introduction

DevOps end to end services & solutions

Agenda

- > ➤ Kubernetes (K8s) History
- > ➤ What is Kubernetes ?
- > ➤ K8s Architecture
- > ➤ Hands-On

History of Kubernetes (K8s)

In 2014

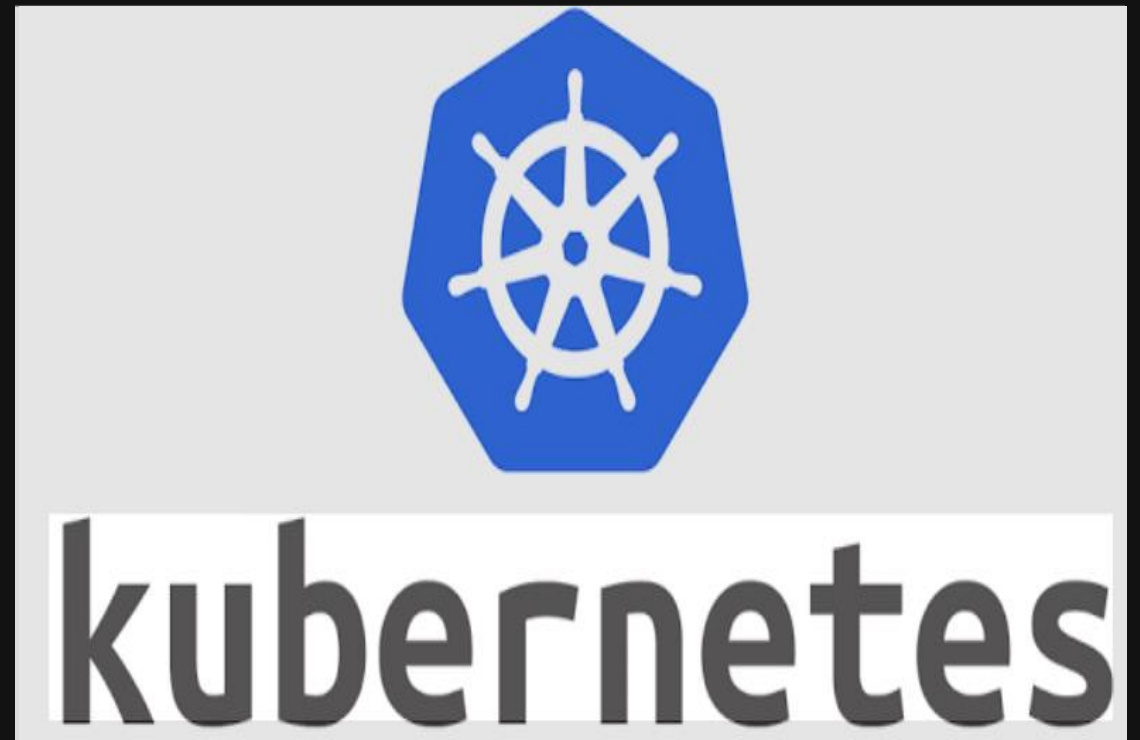
Kubernetes was developed by engineers at Google as an open-source container orchestration system

Google had already been using an internal system called Borg to manage its containerized workloads at scale

It was designed to be cloud-agnostic, unlike Borg, which was specific to Google's infrastructure

Kubernetes, often abbreviated as K8s (where "8" represents the eight letters between "K" and "s"), was born out of this experience

Container orchestration system



History of Kubernetes (K8s)

In 2015

Kubernetes was the first project that graduated Cloud Native Computing Foundation (CNCF)

CNCF was formed under the Linux Foundation to promote containerized application development and ecosystem growth

2015 (Version 1.0):

Kubernetes reached its first stable release (Version 1.0) in July 2015.

The project gained early adopters, including Red Hat and CoreOS

Container orchestration system



History of Kubernetes (K8s)

In 2018

K8s achieved General Availability (GA) status for many core features, such as workloads and networking.

The community introduced Custom Resource Definitions (CRDs), allowing developers to extend Kubernetes functionality.

Helm (a package manager for Kubernetes) gained widespread adoption

K8s became the dominant container orchestration tool

Major cloud providers, including Google, Amazon (AWS), and Microsoft (Azure), introduced Kubernetes support in their platforms

Container orchestration system



What is Kubernetes ?

Core Concept

- Kubernetes is an open-source platform for managing containerized orchestration
- Allow to run any kind of workloads on any cloud or on-prem
- K8s responsible for maintaining the desired state

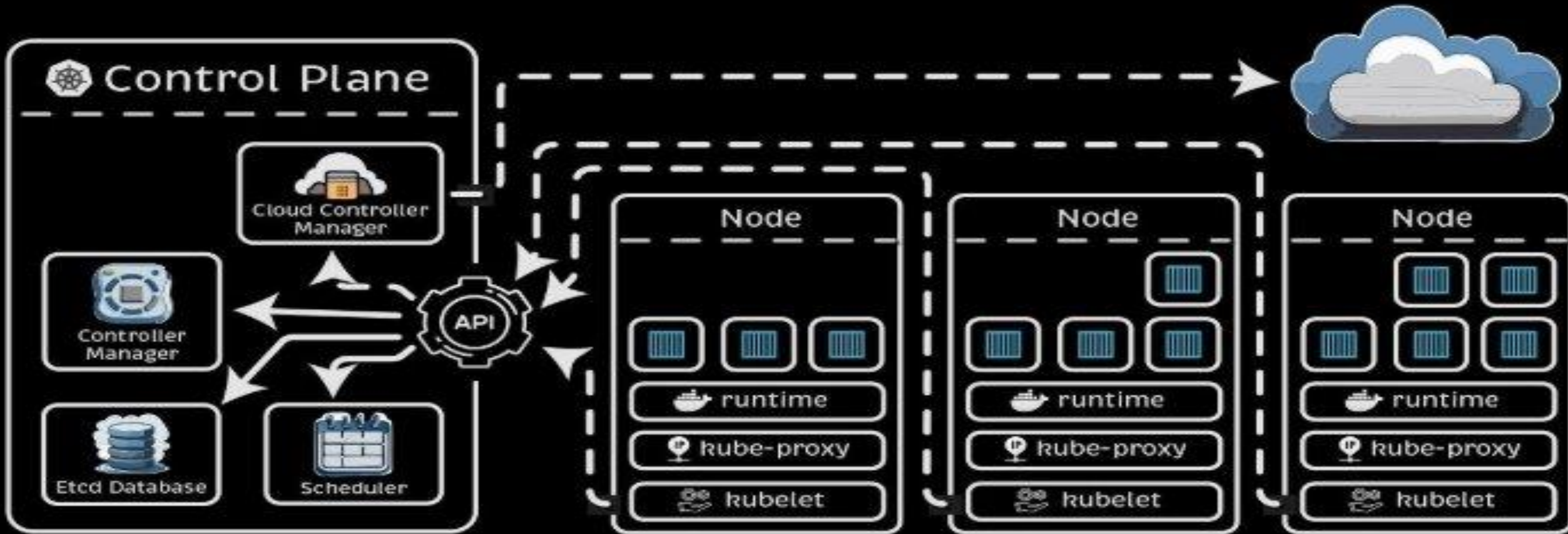
Common Usage

- Providing scalability, reliability, and agility
- Automates deployment and management.
- K8s Self-healing capabilities and is Fault tolerance
- Service discovery and load balancing
- Declarative configuration and automation

Treat and encourages K8s clusters as cattle, not pets

K8s Architecture

Kubernetes Architecture

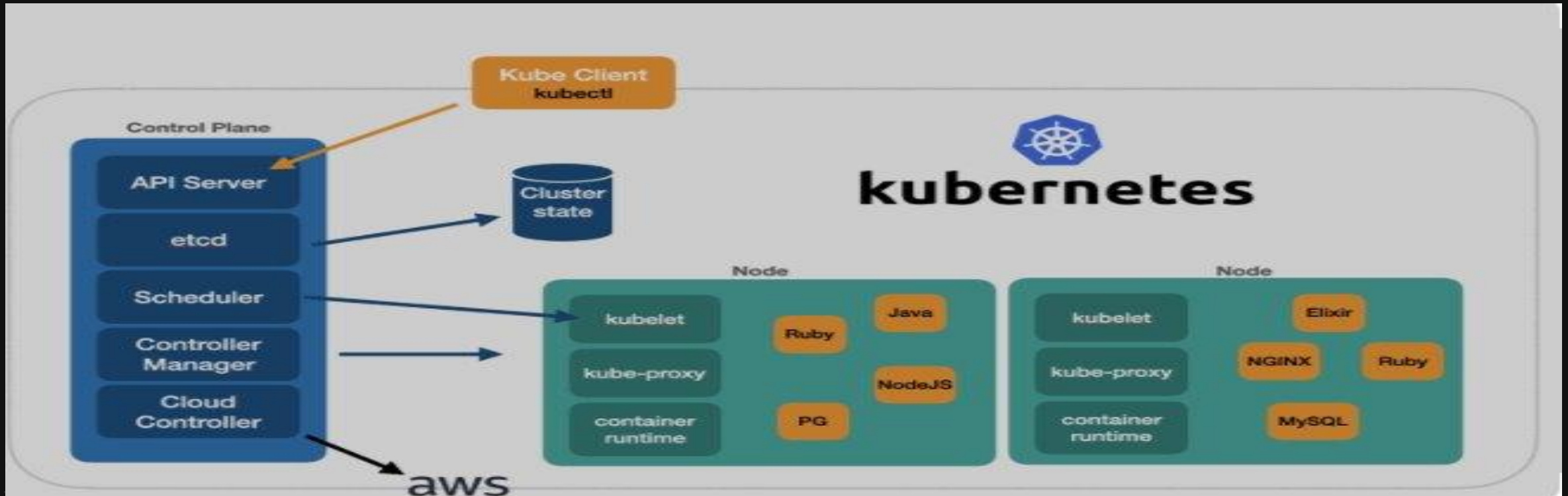


K8s Architecture

Diagram: Include a high-level architecture diagram showing:

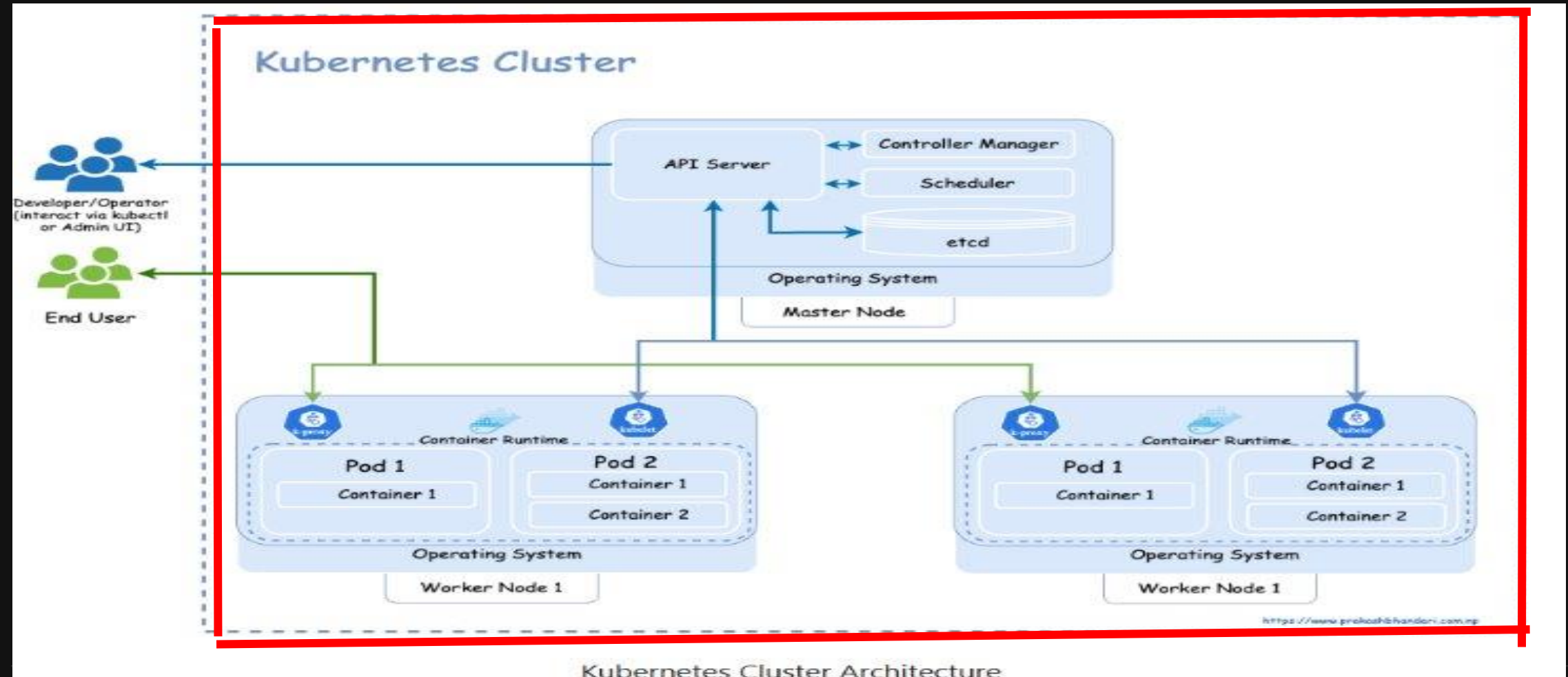
Control Plane: API Server, Scheduler, Controller Manager, etcd.

Nodes: Kubelet, Kube Proxy, Pod, Container Runtime.



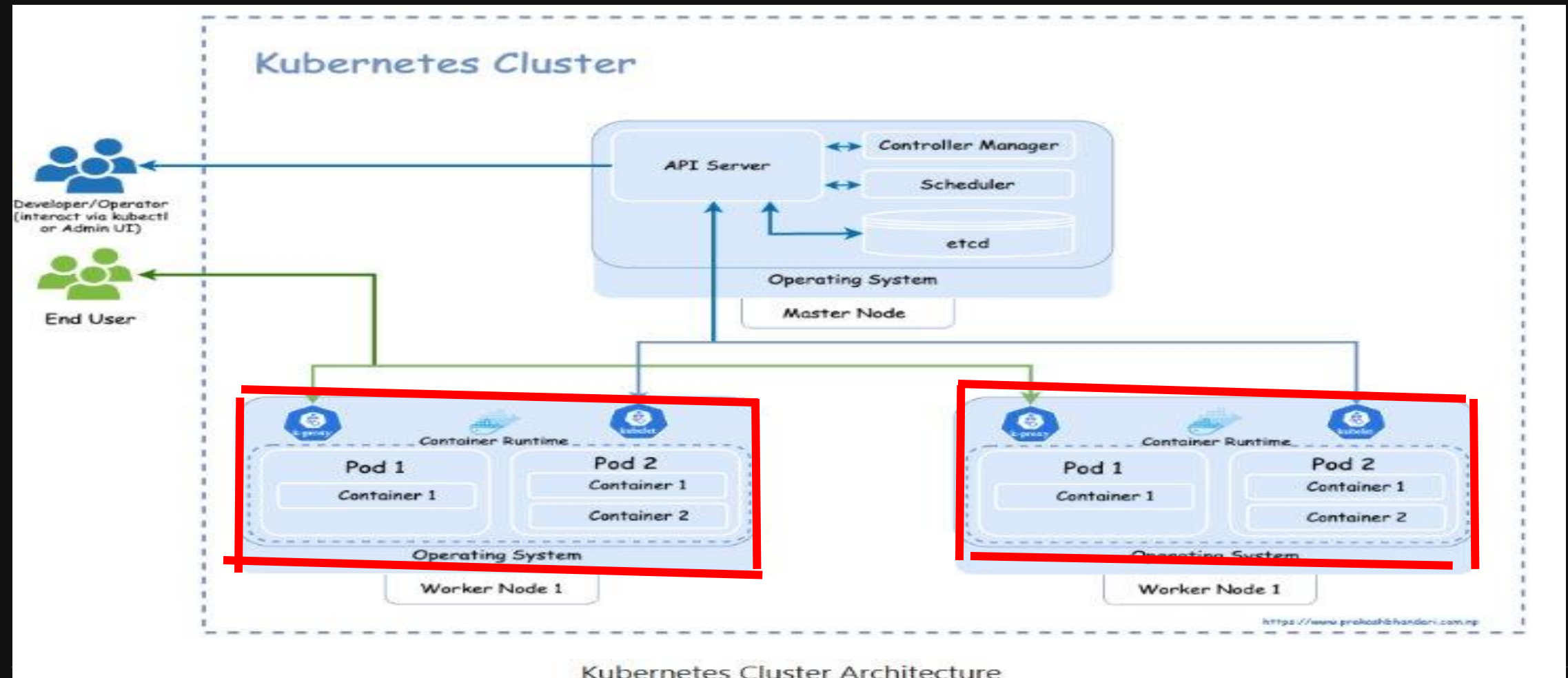
K8s Architecture

Cluster: A set of machines running Kubernetes.



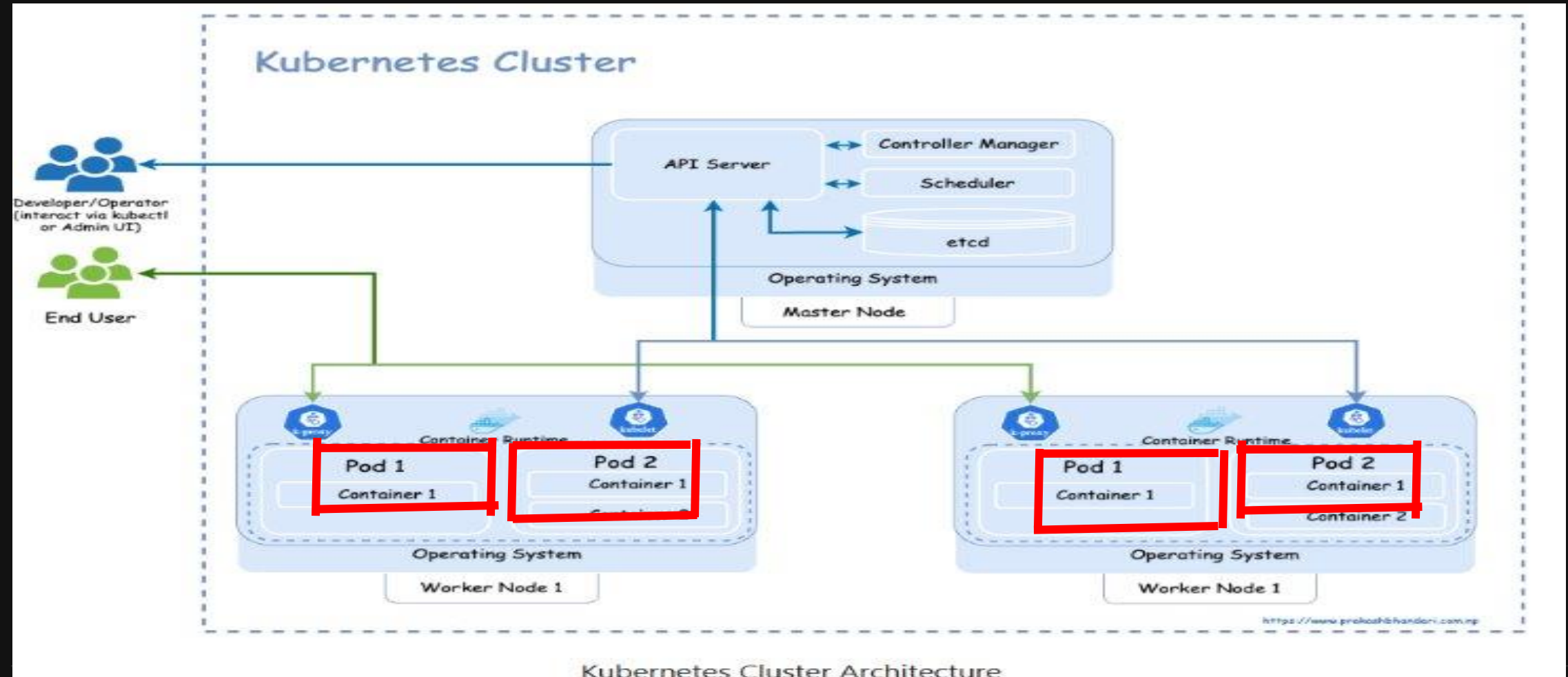
K8s Architecture

Node: A machine (virtual/physical) in the cluster



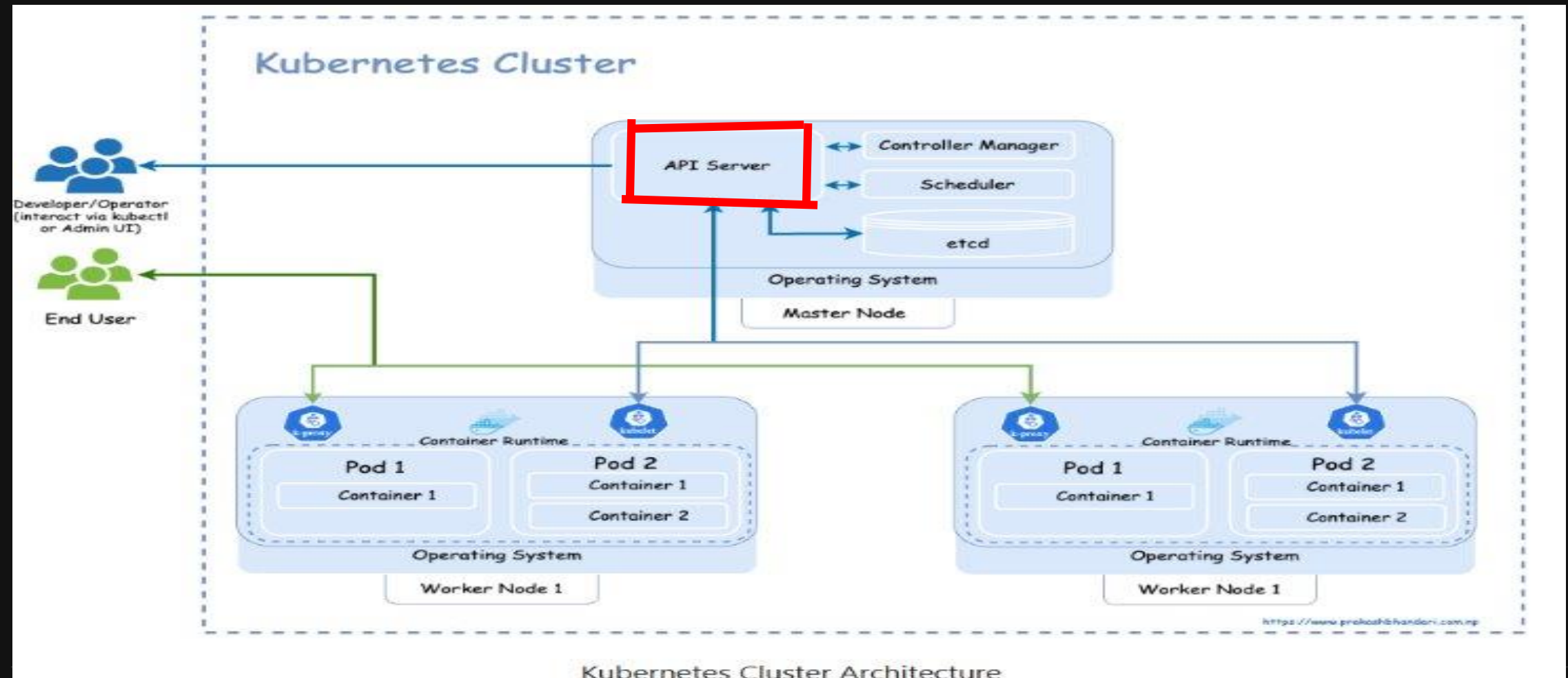
K8s Architecture

Pod: The smallest deployable unit in Kubernetes, Container run time is running Inside the pod



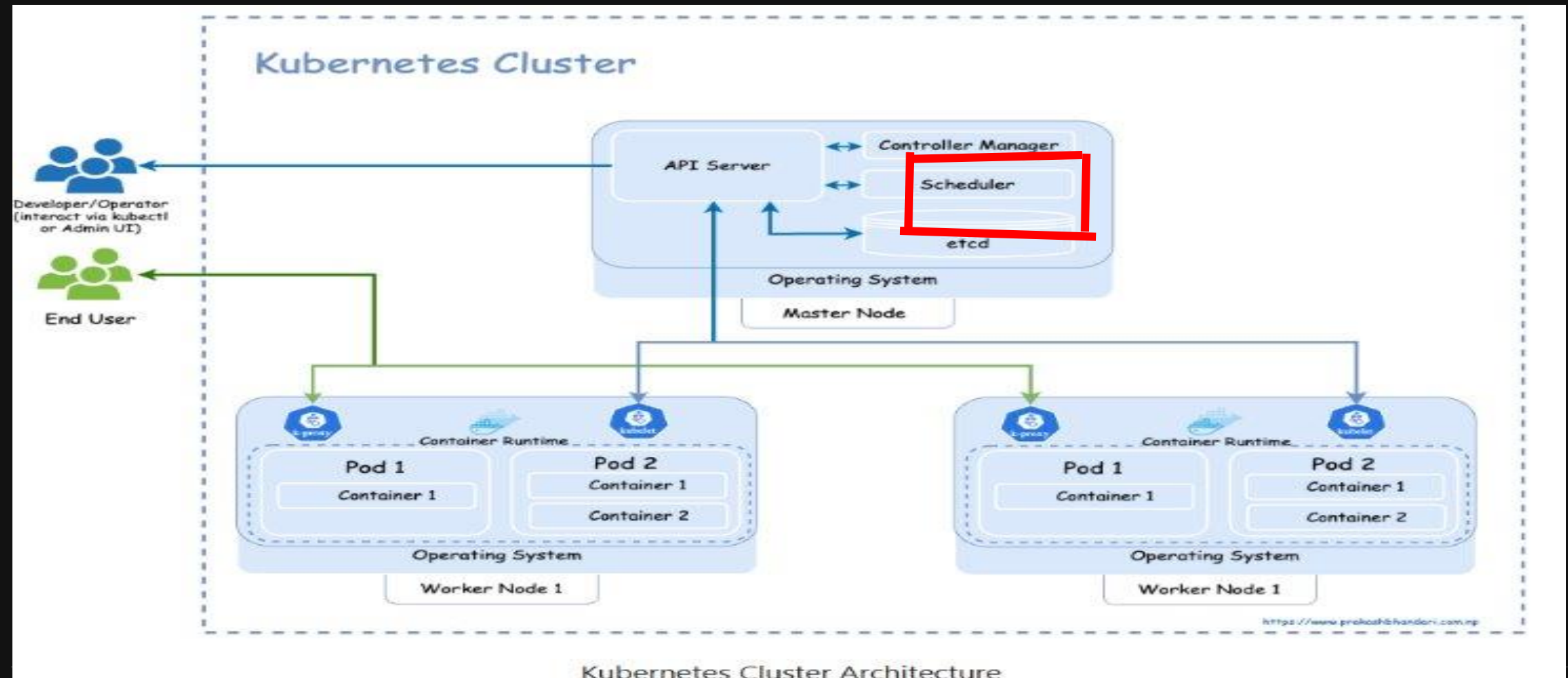
K8s Architecture

API Server: Manages communication as API gateway



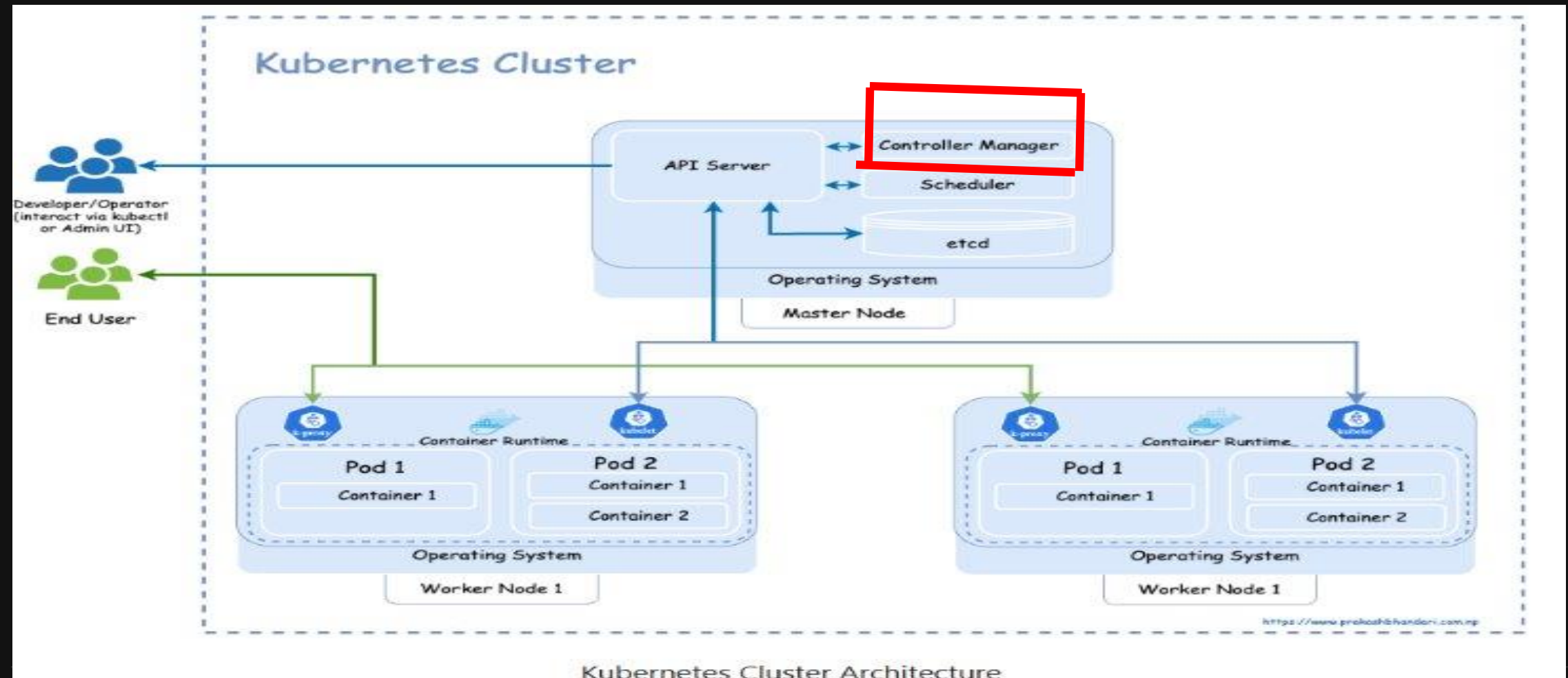
K8s Architecture

Scheduler: Manages and Assigns workloads, distribute the pods for all nodes correctly



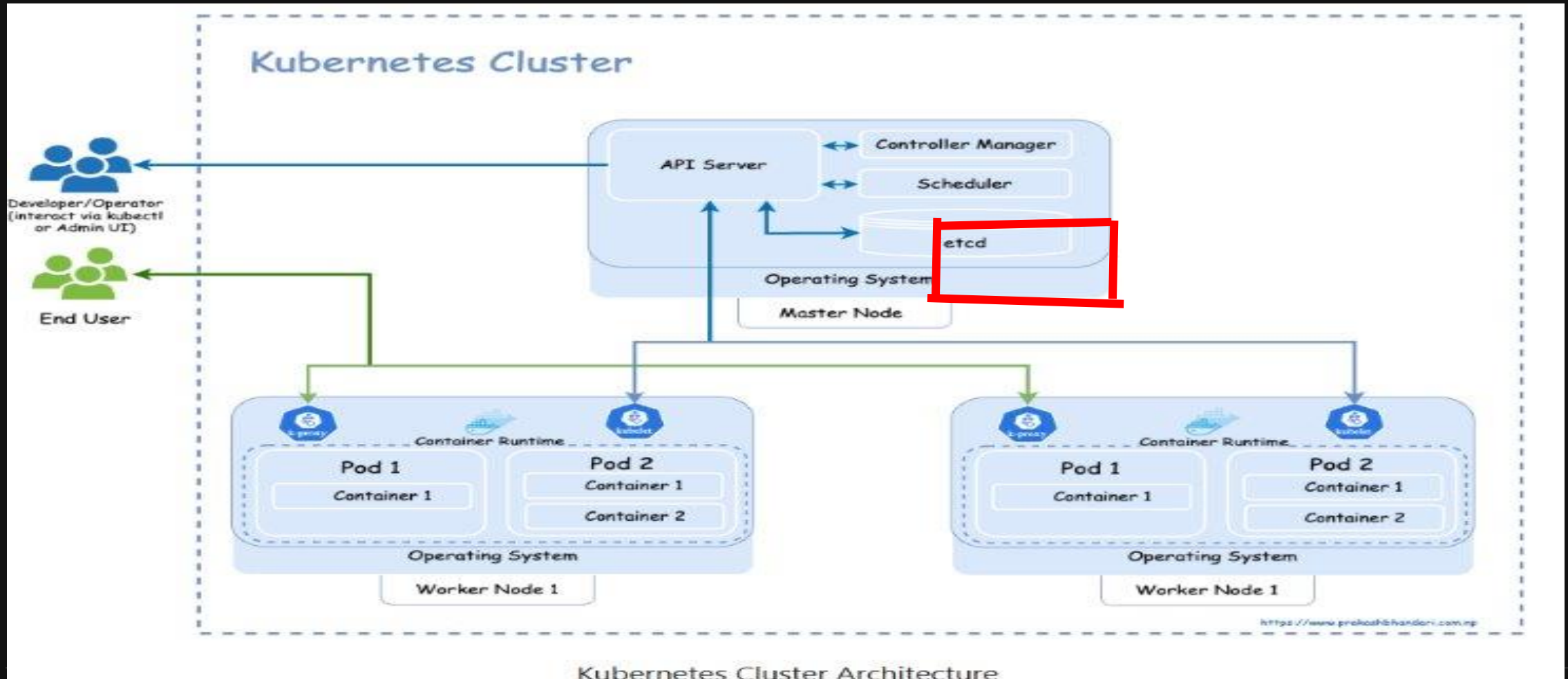
K8s Architecture

Controller Manager: Ensures the desired state of the cluster (check error 300,404,etc)



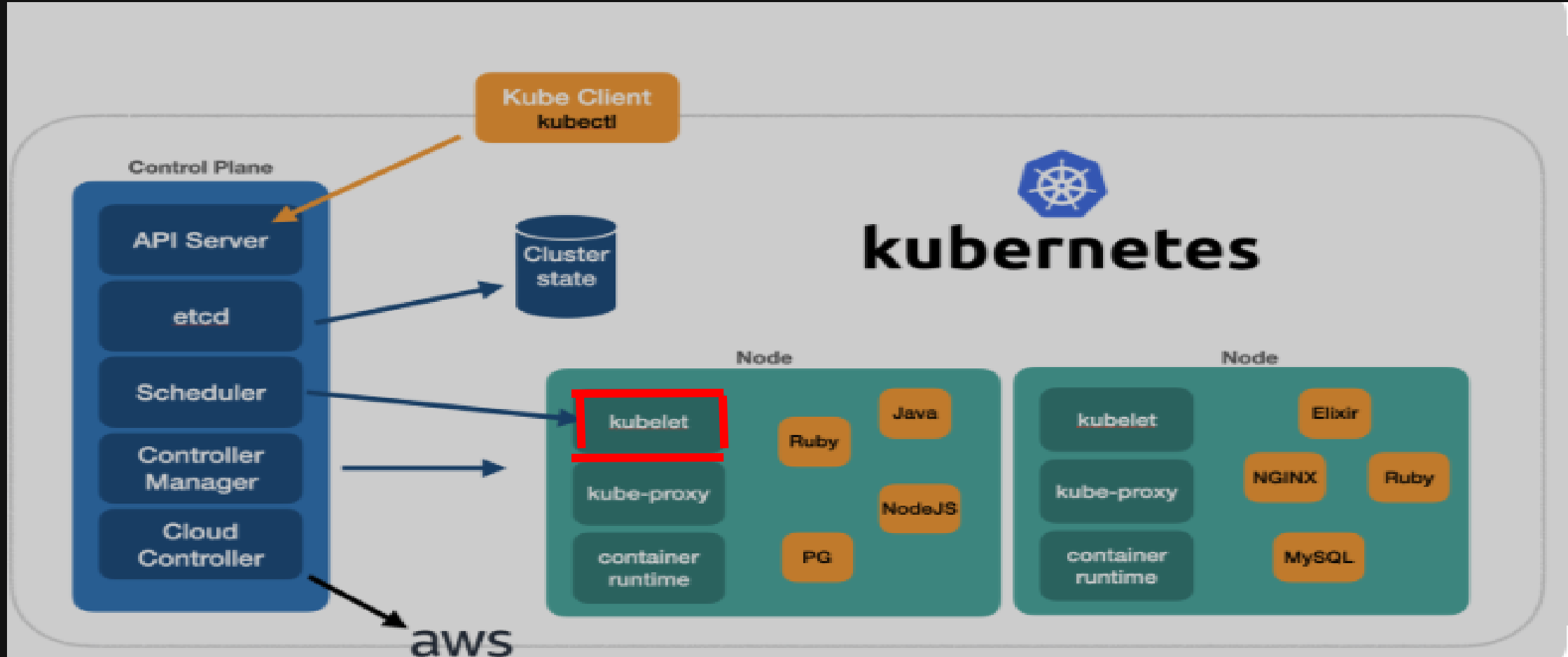
K8s Architecture

ETCD: Ensures the desired state is an open source distributed key-value store used to hold and manage the critical information that distributed systems, means Distributed /etc from linux sys conf files



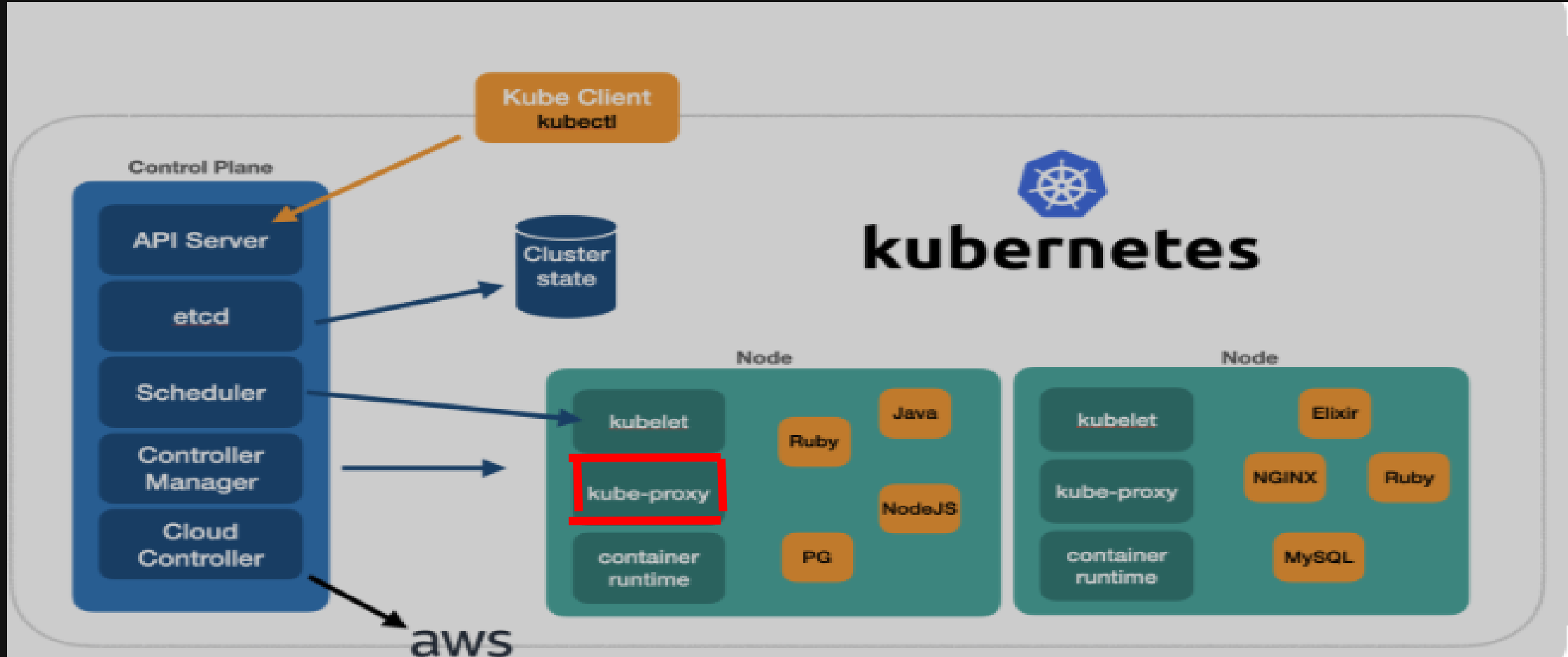
K8s Architecture

Kubelet: Runs containers on a node use as agent service



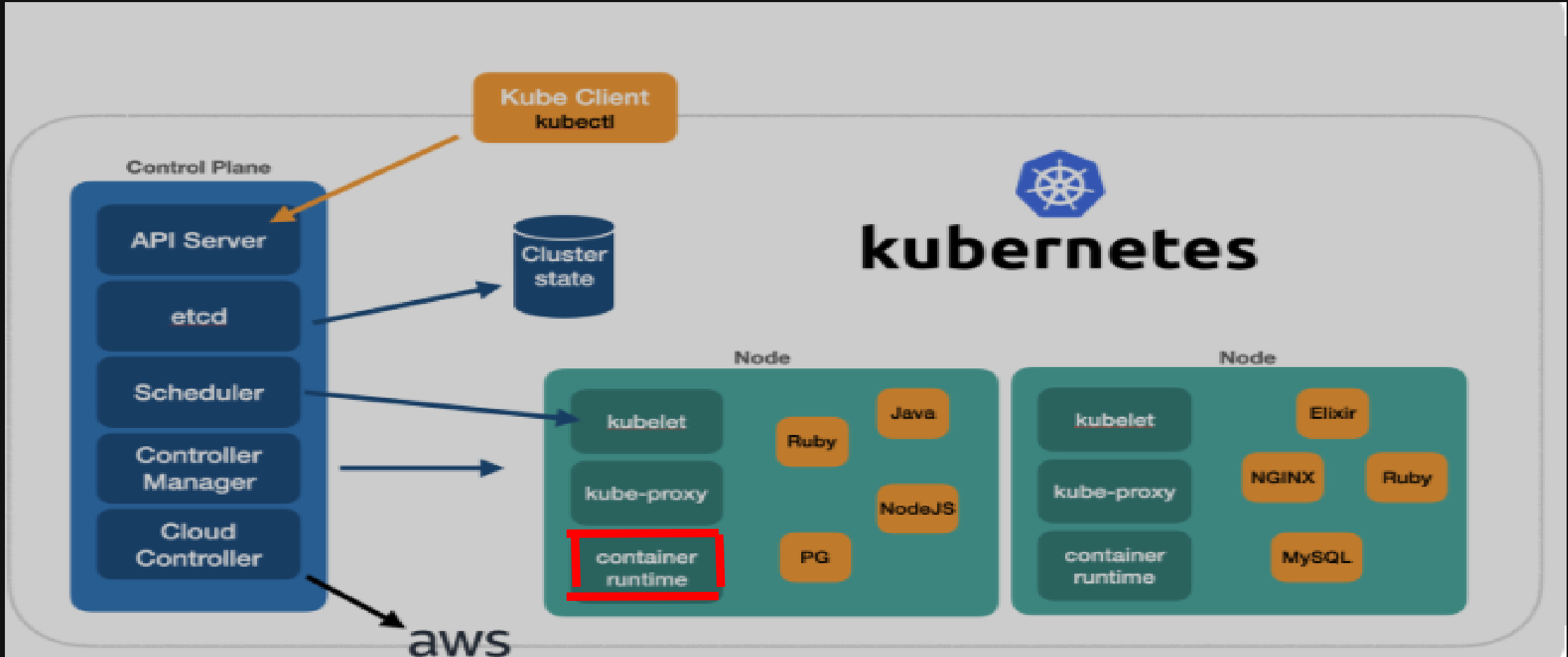
K8s Architecture

Kube Proxy: Manages Networking



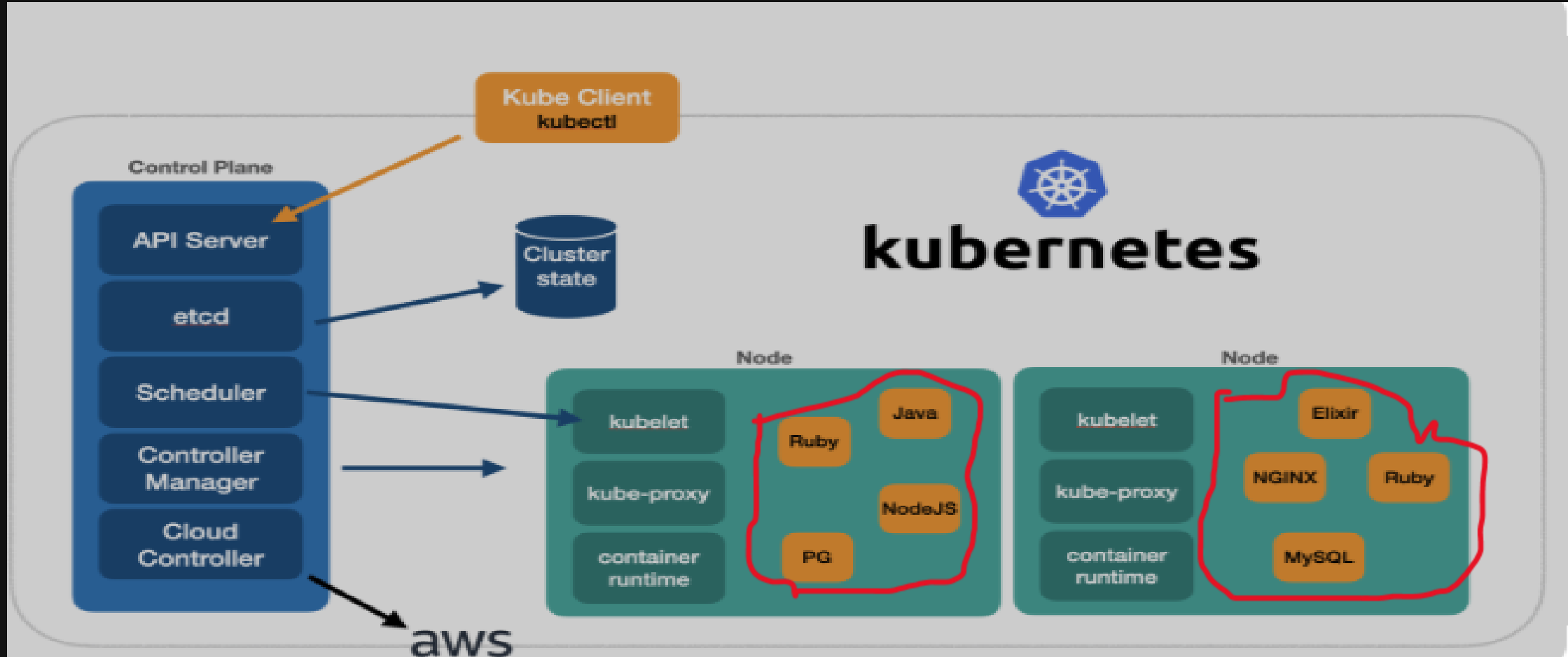
K8s Architecture

Container Runtime Interface (CRI): Is pulling container images from a container registry



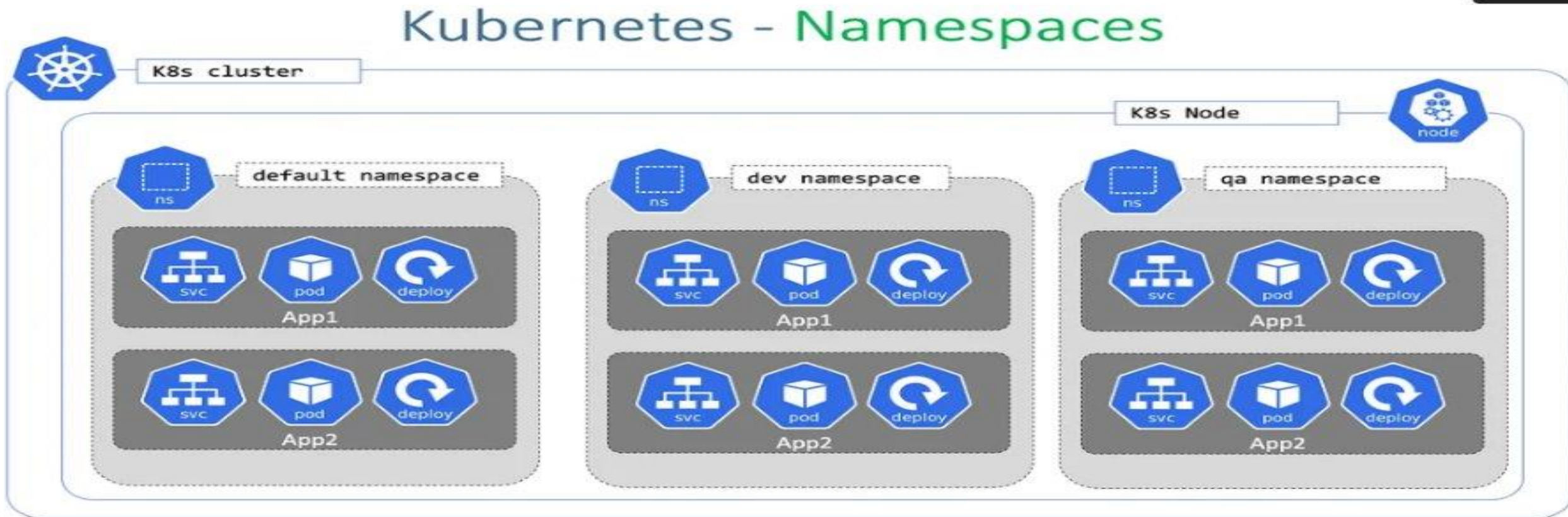
K8s Architecture

Pod: Encapsulate one or more containers



K8s Architecture

Namespaces: are a way to organize clusters into virtual sub-clusters — they can be helpful when different teams or projects share a Kubernetes cluster



Hands-On – Lab 01

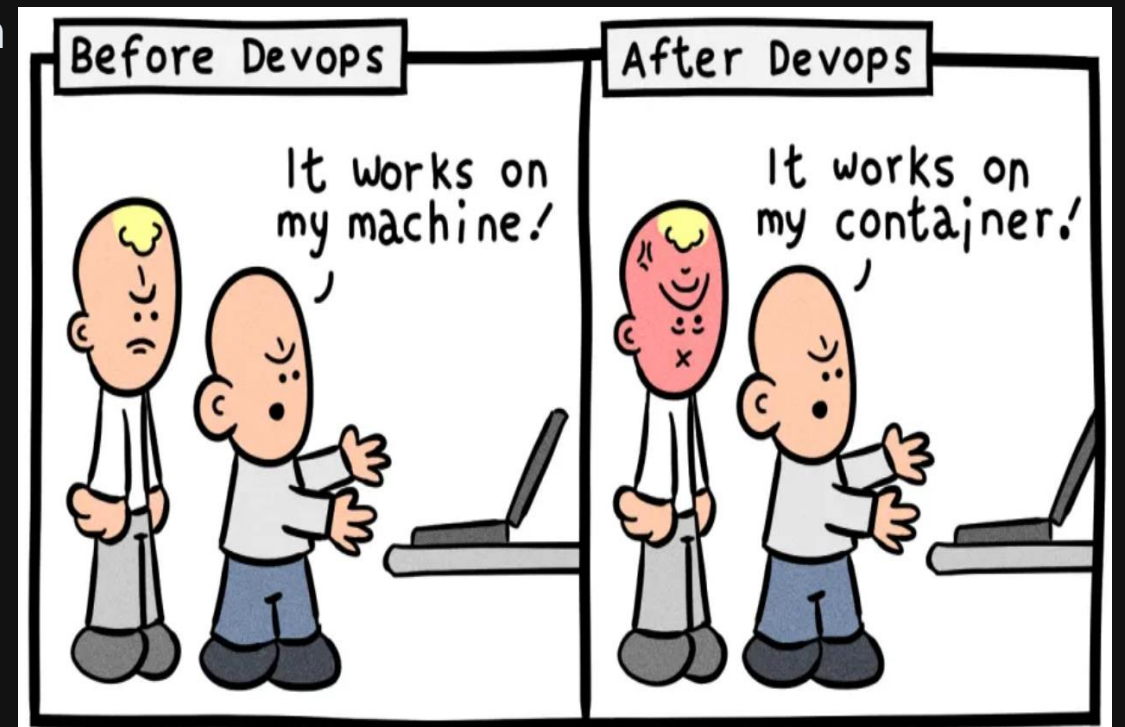
Getting Started

- > ➤ Create namespace with name ns-my-first-pod based on exist cluster
- > ➤ Create yaml file my-first-pod.yaml based on
 - image latest of nginx, Apply the pod

> Bonus

How can we get the IP and Node that the pod running ?

How can we get the log of pod ?



Questions?

Thank you for participating.

Feel free to reach out:
<https://dinghy-e2e.com>

